Technical White Paper

AI Visibility Platform

End-to-end stack, data pipeline, and calculation methodology

Document version 1.0 · April 2026

ENGINEERING & PRODUCT REFERENCE

AI Visibility Platform — Technical White Paper ## End-to-end stack, data pipeline, and calculation methodology
Document version: 1.0
Last updated: April 2026
Audience: Engineering, product, and technical stakeholders

Related documents:

- [Scoring & sentiment methodology](Scoring & Sentiment Methodology (PDF)) — formulas, weights, and UI vs engine consistency

---

1. Executive summary

This platform measures how a **target brand** appears across **consumer-facing AI surfaces** (Perplexity, Google AI / ChatGPT-style results, Gemini). For each scan it:

1. Issues a **structured prompt matrix** (20 prompts × 3 engines = 60 **runs** per full scan).
2. Retrieves answers via a **web-scraping / SERP API** (Infatica), then **parses** HTML or plain text into structured **runs** (mentions, entities, citations, sentiment labels).
3. **Scores** reach (visibility), **share of voice**, **position**, **sentiment**, and a **composite AI Presence Index** using deterministic rules in a shared **scoring engine**.
4. Persists results (JSON + raw runs) and renders **dashboards** (React) with charts, tables, and optional **LLM-generated** narrative insights.

The methodology is **transparent and reproducible**: numeric outputs are derived from explicit code paths, not opaque model scores—except where a secondary **Gemini** pass summarizes scan rows for narrative insights.

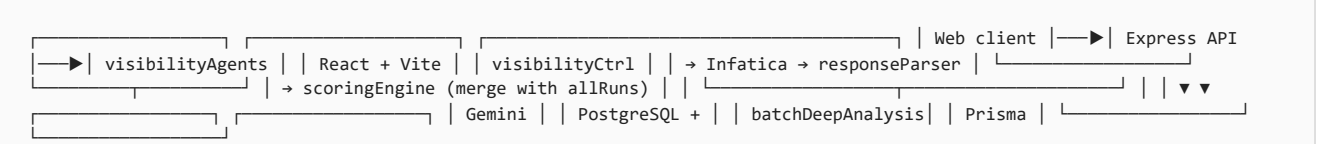
---

2. Product scope and definitions

Term	Meaning
Prompt	One natural-language query from the prompt matrix (`promptIntelligence.js`), with metadata: `id` , `category` , `intent` , optional `weight` .
Engine	One AI answer channel: `perplexity` , `gemini` , or `googleAI` (ChatGPT / Google AI Overview style results as rendered in SERP).
Run	One **prompt × engine** execution: a single row in `allRuns` with `brandMentioned` , `brandEntity` , `entities[]` , `citations[]` , `engine` , `query` , etc.
Mention	Count of occurrences of a brand (or alias) in the **extracted answer text** for that run; stored on each **entity** row (`mentions` or `mentionCount` in some payloads). SOV uses **sum of mentions**, not binary presence.
brandEntity	The target brand's primary entity object for that run (snippet, `positionRank` , `sentiment`). Used for composite sentiment and SOV target sentiment when present.
Citation	A linked source (URL/domain) attached to the answer, capped (e.g. first 15 per run) for storage and analytics.

3. System architecture

Figure 1 — System architecture (data flow)



Flow: User starts a scan → controller creates a `VisibilityScan` record → `runAllAgentsInParallel` queries three engines in **two phases** (first 10 prompts, then remainder) → each response is **parsed** → all runs are **merged** → **scores** and aggregates computed → results JSON stored; optional **deep analysis** enriches the payload.

4. Technology stack

4.1 Frontend (`package.json` — project **searchlyst**)

Layer	Technology
Runtime / bundler	**Vite 6**, **React 18**
Routing	**react-router-dom**
Styling	**Tailwind CSS**, **Radix UI**, **class-variance-authority**
Data fetching	**TanStack React Query**
Charts / maps	**Recharts**, **react-simple-maps**
Forms / validation	**react-hook-form**, **Zod**
Auth / payments	**Google OAuth**, **Stripe** (where enabled)
Misc	**Framer Motion**, **Lucide** icons, **date-fns**

Client-side analytics helpers (examples):

- `src/lib/visibilityTrend.js` — daily/weekly **visibility trend** series from scan history (carry-forward daily, weekly means).
- `src/lib/urlContentType.js` — **domain → content-type bucket** for “Sources” pie (separate from server `categorizeDomain` heuristics; used for consistent UI labeling).

4.2 Backend (`backend/package.json` — searchlyst-backend)

Layer	Technology
Server	**Node.js (ES modules)**, **Express 4**
ORM / DB	**Prisma 5**, **PostgreSQL** (production); adapters also support SQLite paths where used
HTML / text	**Cheerio** for DOM parsing
LLM (analysis)	**@google/generative-ai** (e.g. Gemini 2.5 Flash in `responseParser` for deep analysis)
Jobs / cache (optional)	**Bull**, **ioredis** (in dependencies; use depends on deployment)
Scraping API	**Infatica** HTTPS API (`infaticaService.js`)

4.3 External services

Service	Role
Infatica	Fetches rendered pages / API-shaped payloads for Perplexity, Gemini, and Google AI surfaces; timeouts, retries, query length caps documented in `infaticaService.js` .
Google Generative AI	Optional **batch executive brief** over summarized run data (`batchDeepAnalysis`).
PostgreSQL	Stores users, projects, `VisibilityScan` rows (`results` , `allRuns` , `progress` , `status`).

---

5. End-to-end scan pipeline

5.1 Prompt generation

- **Primary:** `generatePromptMatrixForPlatform` / related logic in `promptIntelligence.js` builds a **Super-20** style matrix: categories such as visibility, ranking, share of voice, geo, deep probe, etc.
- **Fallback:** `generateFallbackPrompts` ensures 20 structured prompts with `weight` , `includesBrand` , and metadata.
- Prompts are **compressed** to fit Infatica query limits (see `MAX_QUERY_LEN`).

5.2 Two-phase parallel execution (`visibilityAgents.js`)

Phase	Content
Phase 1	First **10** prompts, all **3** engines in parallel → ~30 runs; **early results** callback persists partial overview for faster UI.
Phase 2	Remaining **10** prompts → merged with Phase 1 → **60** runs total for a full matrix.

Per-engine batching reduces wall-clock time: e.g. `BATCH_SIZES` (Perplexity 2, Gemini 3, Google AI 3 concurrent prompts per batch). Progress callbacks throttle DB updates (~2s).

5.3 Response acquisition (`infaticaService.js`)

- Engine-specific query functions: `queryPerplexity` , `queryGemini` , `queryGoogleAI` .
- **Retries** on 5xx, **~35s timeout** per attempt, query truncation.
- Returns `{ text, html, sources }` (shape may vary by engine); parser normalizes.

5.4 Parsing (`responseParser.js`)

Entry: `parseResponse(infaticaResult, brandName, domain, competitors, engine)` .

1. **Prefer plain** `text` when present: merge structured `sources` with **regex-extracted** URLs / "Sources:" sections (`extractSourcesFromAIText`).
2. **Else** `html` : `fastParse` uses **engine-specific Cheerio selectors** (`extractAIAnswerFromRenderedPage`) to isolate the main answer, then falls back to full-body text.
3. **Links:** `extractLinksFromHtml` builds citation candidates (with Google redirect unwrap).
4. `buildRunData(text, sources, …)` :
    - **Aliases** (`getAliases`): brand name, domain, domain stem, cleaned corporate suffixes.
    - **Mentions:** `checkMentions` counts alias hits and earliest position in lowercased text.
    - **Entities:** Target brand + each competitor with `mentions > 0` , sorted by **first appearance**; `positionRank` = 1-based order among co-mentioned brands in that answer.
    - **Sentiment (source signal):** `quickSentiment` — local lexical window around brand mention (see §6).
    - **Citations:** Up to **15** per run; `categorizeDomain` assigns `owned` / `competitor` / forum / social / review / institutional / editorial / other.
5. **Storage cap:** `rawText` may be truncated with head/tail preservation (`capRawTextForStorage` , ~8k chars).

Failed or empty responses still produce a run with `brandMentioned: false` and empty entities where appropriate so denominators stay consistent.

5.5 Aggregation and API assembly (`visibilityController.js`)

`buildOverviewFromPlatforms` / `assembleResult` :

- Calls `scoringEngine` functions on `allRuns` (full scan).
- Builds **per-prompt** maps with engine-level sentiment, snippets, citations.
- Attaches **query tracking**, **URL ranking**, **source domains**, **industry presence ranking**, **competitor gaps**, **score** object, **share of voice**, etc.
- **Deep analysis:** `batchDeepAnalysis` runs **after** structured data exists; failure is non-fatal.

---

6. Upstream signals: mention detection and sentiment

6.1 Mention detection

- **Case-insensitive** matching over extracted answer text.

- **Multi-alias** support reduces under-counting (brand string + domain variants).
- **Occurrence count** feeds SOV and entity rows; **first position** drives **position rank** among brands in the same answer.

6.2 Sentiment label (`quickSentiment`)

- **Not** a separate ML classifier in the hot path.
- A **fixed lexicon** of positive and negative English tokens is counted in a **±150 character window** around the first brand mention.
- Rules: if `pos > neg + 1` → `positive`; if `neg > pos + 1` → `negative`; else `neutral`.
- Lexicon samples (see `responseParser.js`): positive includes terms like *best*, *leading*, *recommended*; negative includes *worst*, *avoid*, *scam*, *poor*, etc.

These labels feed `brandEntity.sentiment` and competitor entity sentiment, then flow into **weighted scores** and **breakdown buckets** per [SCORING_AND_SENTIMENT_METHODOLOGY.md](Scoring & Sentiment Methodology (PDF)).

---

7. Scoring engine — parameter catalog

All deterministic metrics live in `backend/src/services/scoringEngine.js`. The following table is the **catalog**; exact formulas and weight rationale are in the linked methodology doc.

Function / output	What it measures	Notes
`computeVisibilityScore`	Composite **AI Presence Index** + components	Visibility %, SOV %, position score, sentiment score; weights **30 / 30 / 20 / 20**.
`computeShareOfVoice`	Mention-volume SOV + table sentiment indices	Target: per-run `brandEntity`; competitors: per-run dominant `entities[]` row.
`computeSentimentBreakdown`	Positive / neutral / negative **counts** and **summary**	Includes `rawCounts` for precise UI KPIs.
`computePerEngine`	Mention rate per engine	Binary appearance rate per engine across runs.
`computePerCategory`	Mention rate per prompt **category**	Uses run `category`.
`computeQueryTracking`	Per-prompt roll-up	Mentioned engines, avg position, citation totals, **sentiment %** per engine + scalar blend (62.5 neutral display alignment).
`computeSourceDomains`	Domain frequency	Aggregates `citations[]` with counts, unique URLs, engine set.
`computeUrlRanking`	URL frequency	Sorted URL list with engine and prompt coverage.
`computeCompetitorGap`	Content opportunities	Prompts where brand **absent** but competitors **present**; generates topic/angle strings.
`computeIndustryPresenceRanking`	**Breadth** vs SOV **volume**	**Prompt coverage %** per entity name + mentions.
`computeIndustryRanking`	SOV-ordered entity list	Wraps `computeShareOfVoice`.

Important: Platform-level summaries in `buildPlatformResults` filter `dataRuns = engineRuns.filter(r => r.textLength > 0)` for some per-engine stats so empty failures do not dilute engine-specific scores—while the **global** overview typically uses all runs; check call sites when interpreting a number's denominator.

---

8. Frontend methodology (derived metrics)

Area	Methodology
Visibility trends	`buildVisibilityTrendDaily`: one point per **calendar day**, **last known score carried forward**; `buildVisibilityTrendWeekly`: **Monday-start week**, score = **mean** of scans in week.
Sentiment & Geo KPI	Weighted blend of breakdown `rawCounts` when available: 100 / **62.5** / 12 for positive / neutral / negative; fallback to percentage fields with small rounding error possible.
Sources pie (UI)	`classifyCitationRow` / `classifyDomainContentType`: **owned**, **competitor**, or heuristic domain buckets (news, forum, social, etc.).

9. Deep analysis layer (LLM)

`batchDeepAnalysis` compresses each run to a **single line** (query, engine, entities, top citation domains) and asks **Gemini** for a **strict JSON** executive brief (overall assessment, strengths, weaknesses, recommendations). This output is **supplemental narrative**; it does not redefine numeric scores unless the product explicitly binds UI to those fields.

---

10. Data persistence and history

- **`VisibilityScan`**: `results` (final JSON for UI), `allRuns` (full run array for replay/debug), `progress` (JSON phase + counts), `status`.
- Historical **trend charts** consume **dated score snapshots** from client/server history APIs (implementation in dashboard services); trend math is client-side in `visibilityTrend.js` as described above.

---

11. Operational characteristics

Topic	Behavior
Latency	Tuned for ~3 minutes full scan (phase batching + parallel engines); network and target sites dominate.
Partial failure	Per-call `Promise.allSettled`; failed runs contribute as empty/zero-mention runs where applicable.
Determinism	Scores are **deterministic** given the same `allRuns` JSON; LLM narrative is not.
Thin data	Composite uses **full component weights** even with few qualifying runs (documented trade-off).

---

12. Limitations and future work

1. **Sentiment** is lexicon-based and English-centric; sarcasm and context are not modeled.
2. **Rendered HTML** changes on Perplexity / Google / Gemini can break selectors; `fastParse` includes fallbacks.
3. **SOV** is only as good as **entity extraction** (configured competitors + target); unknown brands in text are not auto-modeled unless they appear in `entities`.
4. **Geo** sentiment on the dashboard may use **heuristic** region mapping from citation TLDs / domain patterns—not geo-IP of the user.
5. **Weight tuning** (30/30/20/20) is product policy; should be reviewed against calibration studies.

---

13. Code map (quick reference)

Concern	Primary files
Scan orchestration	`backend/src/controllers/visibilityController.js`
Parallel agents	`backend/src/services/visibilityAgents.js`
Fetch + retry	`backend/src/services/infaticaService.js`
Parse + sentiment lexicon	`backend/src/services/responseParser.js`
All numeric scores	`backend/src/services/scoringEngine.js`
Prompts	`backend/src/services/promptIntelligence.js`
Trends (UI)	`src/lib/visibilityTrend.js`
URL / source typing (UI)	`src/lib/urlContentType.js`
Schema	`backend/prisma/schema.prisma`

---

14. Conclusion

The platform combines **controlled data collection** (fixed prompt matrix, three engines), **transparent parsing** (Cheerio + text heuristics), and a **single scoring engine** for KPIs. Narrative insights are an optional **second-stage LLM** layer. For **every formula and sentiment weight**, treat [SCORING_AND_SENTIMENT_METHODOLOGY.md](Scoring & Sentiment Methodology (PDF)) as the authoritative supplement to this white paper.

End of white paper.